

Министерство науки высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

Факультет информационных технологий и программирования

Образовательная программа Программирование и интернет-технологии

Направление подготовки (специальность) 09.04.02 - Информационные системы и технологии

Современная теория информации

О Т Ч Е Т

по лабораторной работе № 1

Студент: Ноздряков Богдан Валериевич

Группа: М4106

Реализованный алгоритм: 4

Преподаватель: Беляев Евгений Александрович

ОГЛАВЛЕНИЕ

ГЛАВА 1 ВВЕДЕНИЕ И ПОСТАНОВКА ЗАДАЧИ	3
1.1. Выбор алгоритма	3
1.2. Среда реализации	3
1.3. Требования к программам	3
1.4. Исходные данные и вычисляемые характеристики	3
1.5. Реализованные версии программы	4
ГЛАВА 2 ОПИСАНИЕ БАЗОВОГО АЛГОРИТМА	4
2.1. Общая идея нумерационного кодирования	4
2.2. Кодирование композиции сообщения	5
2.3. Кодирование последовательности при известной композиции	5
2.4. Формат архива	6
2.5. Декодирование и проверка корректности	6
ГЛАВА 3 РЕАЛИЗОВАННЫЕ УЛУЧШЕНИЯ АЛГОРИТМА	7
3.1. Улучшение №1: более компактное кодирование заголовка	7
3.2. Улучшение №2: блочное нумерационное кодирование	7
ГЛАВА 4 АНАЛИЗ РЕЗУЛЬТАТОВ	8
4.1. Методика	8
4.2. Результаты базовой версии	9
4.3. Результаты версии с улучшением заголовка	10
4.4. Результаты финальной блочной версии	11
4.5. Сравнение версий по таблицам и графикам	11
ГЛАВА 5 СТРУКТУРА ПРОГРАММНОЙ РЕАЛИЗАЦИИ	15
5.1. Основные классы программы	15
5.2. Скрипты сборки и запуска	16
ВЫВОД	17

ГЛАВА 1 ВВЕДЕНИЕ И ПОСТАНОВКА ЗАДАЧИ

1.1. Выбор алгоритма

В данной лабораторной работе был реализован алгоритм №4 – **нумерационное кодирование на основе арифметического кодирования.**

Нумерационное кодирование позволяет использовать информацию о частотах символов всего сообщения. Таким образом, кодовое представление строится не только на основе самих символов, но и на основе структуры сообщения в целом.

1.2. Среда реализации

Весь программный код был написан на языке **Java**. Для сборки и запуска программы требуется установленная **Java версии 16 или выше**. Для удобства запуска были подготовлены **bat-скрипты и архивы JAR**, позволяющие собрать проект, выполнить кодирование, декодирование и автоматический сбор статистики.

1.3. Требования к программам

По условию лабораторной работы требовалось реализовать две отдельные программы: кодер и декодер, запускаемые из командной строки. Кодер должен принимать имя исходного файла и имя архива, а декодер – имя архива и имя восстановленного файла.

Основное требование к декодеру состояло в том, что он должен восстанавливать исходный файл **бит в бит**, то есть без каких-либо искажений или потери данных.

1.4. Исходные данные и вычисляемые характеристики

Эксперименты проводились на тестовом наборе **Calgary corpus**. Для каждого файла тестового корпуса в работе вычислялись оценки:

- $H(X)$
- $H(X|X)$

- $H(X|XX)$

Помимо этого, для каждой реализованной версии программы определялись средние затраты на символ в битах, размер сжатого файла в байтах, а также суммарный размер всех сжатых файлов по всему корпусу.

1.5. Реализованные версии программы

В ходе работы были реализованы три версии архиватора.

Первая версия является базовой и реализует исходный вариант нумерационного кодирования без дополнительных оптимизаций.

Во второй версии было добавлено **улучшение**, связанное с **более компактным кодированием заголовка**.

В третьей, итоговой версии, помимо улучшенного заголовка было реализовано **блочное нумерационное кодирование**, которое **позволяет лучше учитывать локальные изменения статистики по файлу**.

Именно эти три версии далее сравниваются между собой.

ГЛАВА 2 ОПИСАНИЕ БАЗОВОГО АЛГОРИТМА

2.1. Общая идея нумерационного кодирования

Базовая версия архиватора реализует нумерационное кодирование, в котором кодовое представление сообщения естественным образом распадается на две части.

Первая часть описывает **композицию сообщения**, то есть вектор частот всех символов входного файла. Вторая часть кодирует уже саму последовательность символов при условии, что эта композиция известна как кодеру, так и декодеру.

Пусть исходный файл рассматривается как последовательность байтов длины n над алфавитом мощности 256. Сначала по всему файлу подсчитываются частоты всех символов. Полученный вектор частот однозначно задаёт композицию сообщения и используется как основа для

дальнейшего кодирования.

То есть если композиция сообщения известна, то последовательность можно кодировать среди всех последовательностей с такой же композицией.

2.2. Кодирование композиции сообщения

В базовой версии композиция кодируется как номер соответствующего разбиения числа n на 256 неотрицательных частей. То есть вместо явной передачи всех частот передаётся номер композиции в упорядоченном множестве всех возможных композиций данной длины.

Для этого используется процедура ранжирования композиции на основе биномиальных коэффициентов. На стороне декодера выполняется обратная операция – восстановление композиции по её номеру.

Такой способ позволяет передавать не сам массив частот, а более компактное представление, что особенно важно для файлов большой длины.

2.3. Кодирование последовательности при известной композиции

После того как композиция сообщения известна, кодируется уже сама последовательность символов. Для этого используется арифметическое кодирование.

На каждом шаге вероятность текущего символа определяется как отношение числа его ещё неиспользованных вхождений к общему числу символов, которые остаются незакодированными. Если обозначить через f_i текущее оставшееся число вхождений символа i , а через N – число ещё незакодированных символов, то вероятность текущего символа задаётся выражением:

$$p(i) = \frac{f_i}{N} \quad (1)$$

После кодирования символа его частота уменьшается на единицу, а величина N также уменьшается. Таким образом, модель постепенно

изменяется по мере продвижения по сообщению. То есть кодовое слово определяется последовательным сужением интервала в соответствии с вероятностями символов.

2.4. Формат архива

В базовой версии архив содержит служебную информацию, необходимую для последующего восстановления данных, описание композиции сообщения и арифметически закодированную полезную нагрузку.

В заголовке сохраняются сигнатура формата, длина исходного файла и данные, необходимые для восстановления композиции. После этого в архив записывается арифметически закодированная последовательность символов.

Такой формат позволяет декодеру полностью восстановить вероятностную модель и затем выполнить обратное декодирование без потери информации.

2.5. Декодирование и проверка корректности

Декодирование выполняется в обратном порядке. Сначала из архива считывается служебный заголовок, затем восстанавливается композиция сообщения. После этого декодер последовательно извлекает символы из арифметически закодированного потока, используя те же остаточные частоты, что и кодер.

Корректность работы программы проверялась автоматически. После декодирования полученный файл сравнивался с исходным сначала по размеру, а затем побайтно. Если на любом этапе обнаруживалось расхождение, программа завершалась с исключением, и такой результат не включался в итоговую статистику. Для всех файлов, вошедших в итоговые результаты, было получено полное совпадение. Это означает, что разработанный декодер действительно восстанавливает исходные данные **бит в бит**.

ГЛАВА 3 РЕАЛИЗОВАННЫЕ УЛУЧШЕНИЯ АЛГОРИТМА

3.1. Улучшение №1: более компактное кодирование заголовка

Первое улучшение касалось кодирования композиции сообщения.

В базовой версии композиция всегда передавалась одним и тем же способом – через ранговое кодирование. Однако для некоторых файлов, особенно при разреженном распределении символов, такое представление оказывается не лучшим. Поэтому во второй версии был добавлен альтернативный способ кодирования заголовка на случай, когда реально встречается только часть символов алфавита.

В этой версии на этапе кодирования строились два варианта заголовка: обычный ранговый и более компактный sparse-вариант. После этого выбирался тот из них, который требовал меньше бит.

Важно, что это улучшение не меняет полезную нагрузку архива. Оно влияет только на служебную часть, то есть уменьшает число бит, затраченных на описание модели сообщения.

3.2. Улучшение №2: блочное нумерационное кодирование

Второе улучшение связано с переходом от глобального кодирования всего файла одной общей композицией к блочному кодированию.

В итоговой версии файл разбивается на блоки фиксированного размера. Для каждого блока отдельно вычисляется композиция и отдельно выполняется нумерационное кодирование на основе арифметического кодирования.

Идея такого подхода состоит в том, что статистика символов внутри файла может заметно меняться от одной части к другой. Если использовать одну общую модель на весь файл, эти локальные особенности не учитываются. Блочное кодирование позволяет строить более точную модель для каждой части файла и тем самым уменьшать средние затраты на символ.

В программе были рассмотрены несколько размеров блока, после чего для каждого файла выбирался лучший вариант. Именно это улучшение

оказалось наиболее эффективным.

ГЛАВА 4 АНАЛИЗ РЕЗУЛЬТАТОВ

4.1. Методика

Для всех трёх версий архиватора был выполнен прогон по тестовому набору Calgary. Результаты обработки каждого файла выводились в виде таблицы, в которой приведены как статистические характеристики исходного файла, так и параметры, относящиеся к работе архиватора. Столбец **File** содержит имя файла тестового набора. Столбцы **$H(X)$** , **$H(X|X)$** и **$H(X|XX)$** задают оценки одномерной энтропии источника, условной энтропии первого порядка и условной энтропии второго порядка соответственно. Столбец **Size(B)** показывает размер исходного файла в байтах, а **Comp(B)** – размер полученного сжатого файла в байтах. Столбец **Bits/sym** отражает средние затраты на символ, вычисленные по размеру сохранённого архива, а **ExactBits** – точные средние затраты на символ, рассчитанные по фактическому числу записанных бит без учёта выравнивания до целого числа байтов. Столбец **HdrBits** показывает число бит, затраченных на служебную часть архива, а **PadBits** – число добавленных выравнивающих бит в конце потока. Под таблицей дополнительно приводятся **суммарный размер всех сжатых файлов в байтах** и **суммарное число точно записанных бит по всему тестовому набору**.

Дополнительно после каждого запуска кодирования выполнялось декодирование, а восстановленный файл автоматически сравнивался с исходным. В итоговые таблицы включались только корректно обработанные файлы, для которых декодирование давало полное совпадение с исходными данными.

4.2. Результаты базовой версии

File	H(X)	H(X X)	H(X XX)	Size(B)	Comp(B)	Bits/sym	ExactBits	HdrBits	PadBits
bib	5.2007	3.3641	2.3075	111261	72610	5.2209	5.2209	2696	1
book1	4.5271	3.5845	2.8141	768771	435407	4.5309	4.5309	3406	7
book2	4.7926	3.7452	2.7357	610856	366292	4.7971	4.7971	3322	2
geo	5.6464	4.2642	3.4577	102400	72450	5.6602	5.6601	2665	1
news	5.1896	4.0919	2.9228	377109	244948	5.1963	5.1963	3144	2
obj1	5.9482	3.4637	1.4004	21504	16130	6.0007	6.0006	2093	4
obj2	6.2604	3.8704	2.2654	246814	193345	6.2669	6.2669	2989	4
paper1	4.9830	3.6461	2.3318	53161	33361	5.0204	5.0203	2424	5
paper2	4.6014	3.5224	2.5136	82199	47551	4.6279	4.6278	2584	7
paper3	4.6651	3.5548	2.5599	46526	27381	4.7081	4.7080	2375	2
paper4	4.6997	3.4773	2.2052	13286	8007	4.8213	4.8212	1917	1
paper5	4.9362	3.5260	2.0414	11954	7569	5.0654	5.0648	1878	7
paper6	5.0095	3.6112	2.2510	38105	24096	5.0589	5.0588	2302	1
pic	1.2102	0.8237	0.7052	513216	77970	1.2154	1.2154	3258	3
progc	5.1990	3.6034	2.1340	39611	25976	5.2462	5.2462	2316	1
progl	4.7701	3.2116	2.0436	71646	42985	4.7997	4.7996	2534	5
progp	4.8688	3.1875	1.7551	49379	30300	4.9090	4.9089	2397	5
trans	5.5328	3.3548	1.9305	93695	65064	5.5554	5.5553	2633	7
Total compressed size (all files): 1791442 bytes									
Total exact bits (all files): 14331471 bits									

Рисунок 1 — Таблица результатов базовой версии архиватора на тестовом датасете Calgary

По результатам базовой версии видно, что архиватор обеспечивает устойчивое сжатие на всех файлах корпуса и корректно выполняет обратное восстановление данных. При этом средние затраты на символ заметно выше условных энтропий $H(X|X)$ и $H(X|XX)$. Так произошло, потому что базовая версия использует только композицию всего сообщения и не учитывает контекст предыдущих символов.

Алгоритм эффективно использует глобальное распределение символов, но не моделирует зависимости между соседними байтами. Поэтому его результаты логично сравнивать прежде всего с одномерной энтропией источника – $H(X)$.

4.3. Результаты версии с улучшением заголовка

File	H(X)	H(X X)	H(X XX)	Size(B)	Comp(B)	Bits/sym	ExactBits	HdrBits	PadBits
bib	5.2007	3.3641	2.3075	111261	72450	5.2094	5.2093	1412	5
book1	4.5271	3.5845	2.8141	768771	435172	4.5285	4.5285	1527	6
book2	4.7926	3.7452	2.7357	610856	366098	4.7946	4.7945	1767	5
geo	5.6464	4.2642	3.4577	102400	72450	5.6602	5.6602	2666	0
news	5.1896	4.0919	2.9228	377109	244781	5.1928	5.1928	1803	7
obj1	5.9482	3.4637	1.4004	21504	16130	6.0007	6.0006	2094	3
obj2	6.2604	3.8704	2.2654	246814	193345	6.2669	6.2669	2990	3
paper1	4.9830	3.6461	2.3318	53161	33240	5.0022	5.0021	1456	5
paper2	4.6014	3.5224	2.5136	82199	47401	4.6133	4.6133	1390	1
paper3	4.6651	3.5548	2.5599	46526	27243	4.6843	4.6843	1272	1
paper4	4.6997	3.4773	2.2052	13286	7906	4.7605	4.7603	1107	3
paper5	4.9362	3.5260	2.0414	11954	7487	5.0105	5.0105	1229	0
paper6	5.0095	3.6112	2.2510	38105	23984	5.0353	5.0352	1401	6
pic	1.2102	0.8237	0.7052	513216	77815	1.2130	1.2130	2017	4
progc	5.1990	3.6034	2.1340	39611	25865	5.2238	5.2238	1429	0
progl	4.7701	3.2116	2.0436	71646	42838	4.7833	4.7832	1356	7
progp	4.8688	3.1875	1.7551	49379	30173	4.8884	4.8884	1385	1
trans	5.5328	3.3548	1.9305	93695	64939	5.5447	5.5447	1636	4
Total compressed size (all files): 1789317 bytes									
Total exact bits (all files): 14314475 bits									

Рисунок 2 — Таблица результатов версии архиватора с улучшенным заголовком на тестовом датасете Calgary

Первое улучшение дало небольшой, но устойчивый выигрыш по сравнению с базовой версией. Поскольку изменялся только способ представления композиции сообщения, основное уменьшение размера архива произошло за счёт служебной части.

Суммарный размер сжатых файлов уменьшился с **1_791_442 байт** до **1_789_317 байт**, то есть на **2125 байт**. Точное число записанных бит уменьшилось на **16_996 бит**.

Таким образом, улучшение заголовка не меняет принцип кодирования последовательности, но делает описание модели более компактным, что и даёт наблюдаемый выигрыш.

4.4. Результаты финальной блочной версии

File	H(X)	H(X X)	H(X XX)	Size(B)	Comp(B)	Bits/sym	ExactBits	HdrBits	PadBits
bib	5.2007	3.3641	2.3075	111261	72454	5.2097	5.2096	1447	2
book1	4.5271	3.5845	2.8141	768771	435176	4.5285	4.5285	1562	3
book2	4.7926	3.7452	2.7357	610856	364900	4.7789	4.7789	41580	1
geo	5.6464	4.2642	3.4577	102400	72455	5.6605	5.6605	2701	5
news	5.1896	4.0919	2.9228	377109	244785	5.1929	5.1929	1838	4
obj1	5.9482	3.4637	1.4004	21504	15146	5.6347	5.6343	18617	7
obj2	6.2604	3.8704	2.2654	246814	185215	6.0034	6.0034	88638	5
paper1	4.9830	3.6461	2.3318	53161	32936	4.9564	4.9563	4539	4
paper2	4.6014	3.5224	2.5136	82199	47406	4.6138	4.6137	1425	6
paper3	4.6651	3.5548	2.5599	46526	27248	4.6852	4.6851	1307	6
paper4	4.6997	3.4773	2.2052	13286	7910	4.7629	4.7629	1142	0
paper5	4.9362	3.5260	2.0414	11954	7492	5.0139	5.0135	1264	5
paper6	5.0095	3.6112	2.2510	38105	23444	4.9220	4.9219	3497	3
pic	1.2102	0.8237	0.7052	513216	73804	1.1505	1.1504	23848	5
progс	5.1990	3.6034	2.1340	39611	25870	5.2248	5.2247	1464	5
progl	4.7701	3.2116	2.0436	71646	42425	4.7372	4.7372	8228	1
progp	4.8688	3.1875	1.7551	49379	30127	4.8809	4.8808	6488	5
trans	5.5328	3.3548	1.9305	93695	64163	5.4785	5.4784	13693	2
Total compressed size (all files): 1772956 bytes									
Total exact bits (all files): 14183579 bits									

Рисунок 3 — Таблица финальной блочной версии архиватора на тестовом датасете Calgary

Наиболее заметный эффект дало второе улучшение — переход к блочному нумерационному кодированию. В этой версии уменьшение касается уже не только заголовка, но и полезной нагрузки, поскольку для разных частей файла используется более точная локальная статистическая модель.

Суммарный размер сжатых файлов для финальной версии составил **1_772_956 байт**. Это на **18_486 байт** меньше результата базовой версии и на **16_361 байт** меньше результата версии, в которой улучшался только заголовок.

То есть именно блочное кодирование внесло основной вклад в повышение эффективности архиватора.

4.5. Сравнение версий по таблицам и графикам

Для наглядного сравнения трёх версий были построены графики суммарного размера сжатых файлов, абсолютной экономии в байтах, относительного уменьшения размера и уменьшения точных затрат на символ.



Рисунок 4 — График, показывающий суммарный размер сжатых файлов для трёх версий архиватора

График суммарного размера показывает, что каждая следующая версия программы улучшает результат предыдущей, а минимальный общий размер архива достигается в финальной версии.

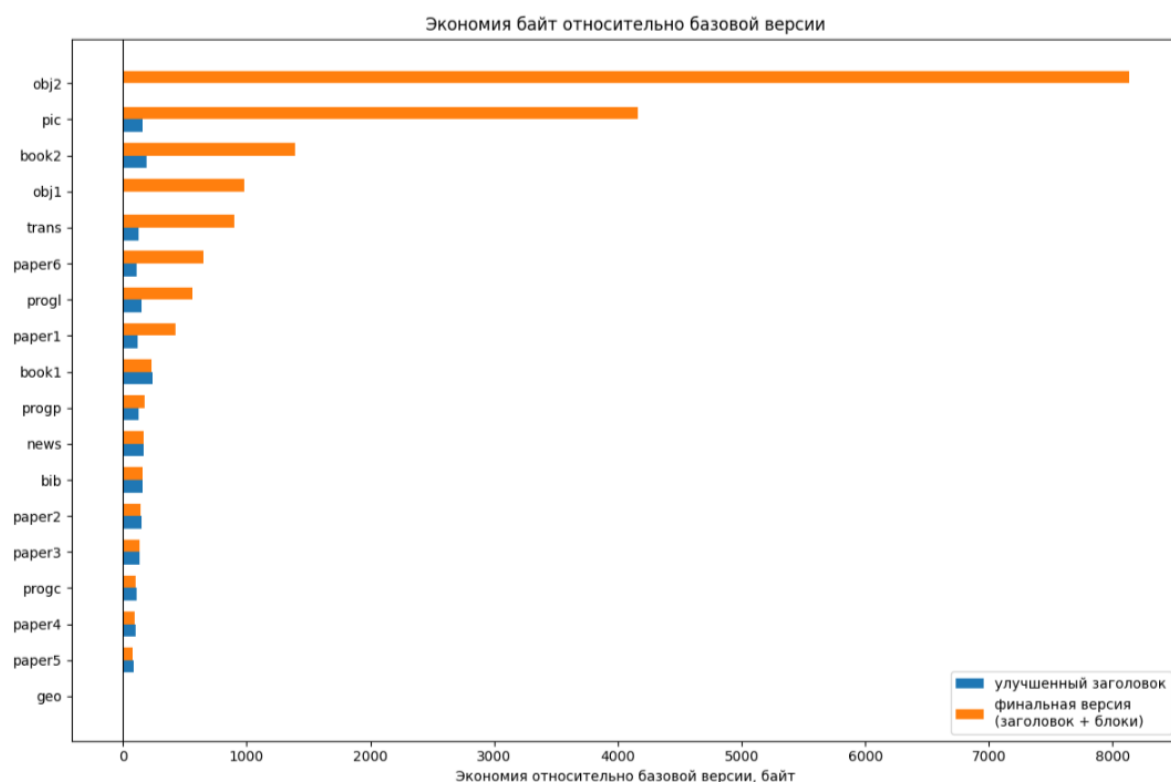


Рисунок 5 — График, показывающий экономию байт относительно базовой версии для файлов Calgary

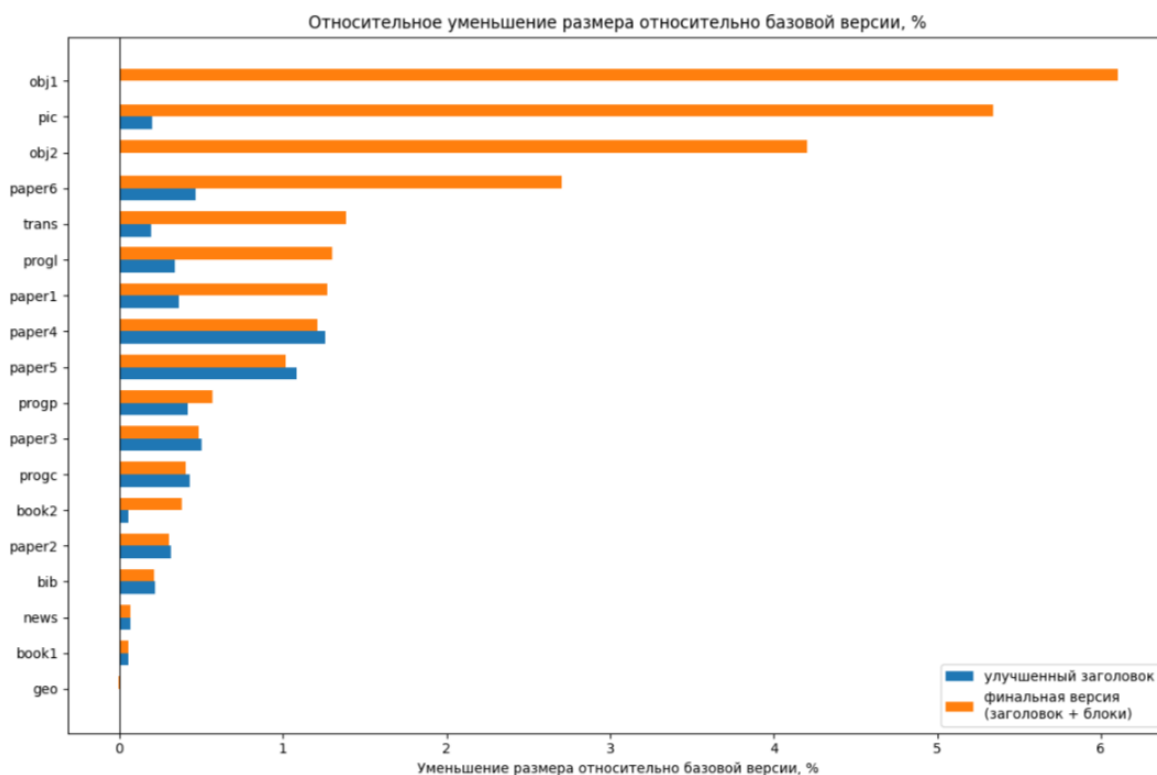


Рисунок 6 — График, показывающий относительное уменьшение размера относительно базовой версии для файлов корпуса Calgary, %

Графики абсолютной и относительной экономии по отдельным файлам показывают, что первое улучшение даёт небольшой выигрыш, тогда как блочное кодирование позволяет сильнее уменьшить размер архива на ряде файлов. Особенно заметный эффект наблюдается на файлах obj2, pic, book2, obj1 и trans.

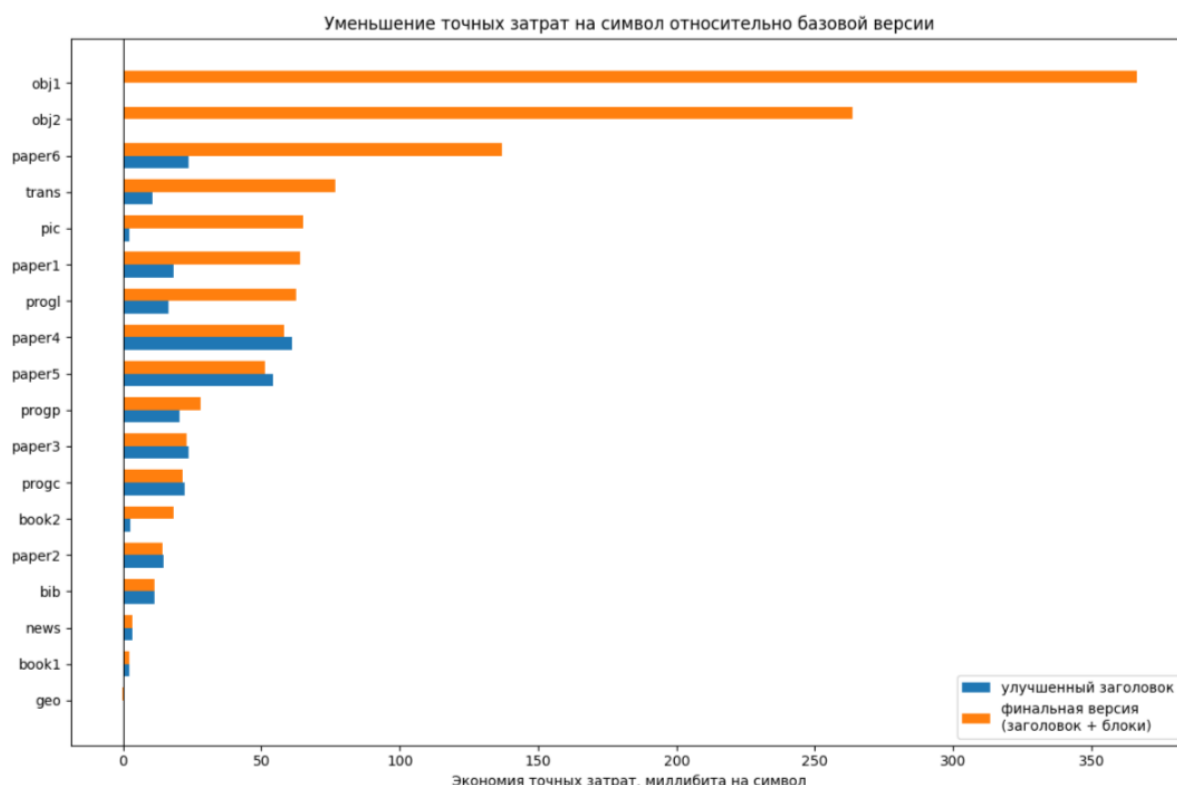


Рисунок 7 — График, показывающий уменьшение точных затрат на символ относительно базовой версии для файлов Calgary

График уменьшения точных затрат на символ подтверждает, что финальная версия лучше использует статистические свойства данных и обеспечивает меньшую стоимость кодирования одного символа.

ГЛАВА 5 СТРУКТУРА ПРОГРАММНОЙ РЕАЛИЗАЦИИ

5.1. Основные классы программы

Классы **BitInput** и **BitOutput** отвечают за побитовый ввод-вывод. Они используются как вспомогательные классы для работы арифметического кодера и для записи архивного формата.

Классы **ArithmeticEncoder** и **ArithmeticDecoder** реализуют арифметическое кодирование и декодирование целочисленным методом с нормализацией интервала.

Класс **CompositionCoder** отвечает за вычисление числа композиций, ранжирование композиции и восстановление композиции по её номеру.

Класс **FenwickTree** используется для быстрого вычисления кумулятивных частот и поиска символа по кумулятивной сумме.

В улучшенной версии был добавлен класс **SparseHeaderCoder**, реализующий более компактный способ кодирования заголовка для разреженных распределений.

Класс **NumerationEncoder** выполняет полный цикл архивирования: чтение исходного файла, подсчёт частот, построение заголовка, кодирование полезной нагрузки и запись архива. Класс **NumerationDecoder** выполняет обратное преобразование и восстанавливает исходный файл.

Класс **StatsRunner** предназначен для автоматического сбора статистики на тестовом наборе: он запускает кодирование и декодирование, проверяет совпадение файлов и собирает статистику, необходимую для таблиц и графиков.

5.2. Скрипты сборки и запуска

Для удобства работы с программой были подготовлены bat-скрипты, позволяющие собирать проект, формировать исполняемые jar-файлы и запускать основные режимы работы из командной строки.

Скрипт **build.bat** выполняет компиляцию всех исходных файлов Java и помещает полученные .class-файлы в каталог out.

Скрипт **build_release.bat** формирует исполняемые jar-архивы encoder.jar, decoder.jar и stats.jar в папке release. Эти jar-файлы содержат все необходимые скомпилированные классы программы и используются как готовые исполняемые модули для запуска кодирования, декодирования и расчёта статистики.

Скрипт **encoder.bat** запускает кодер на основе encoder.jar и создаёт сжатый файл.

Скрипт **decoder.bat** запускает декодер на основе decoder.jar и восстанавливает исходный файл из архива.

Скрипт **stats.bat** запускает модуль статистики на основе stats.jar,

выполняет автоматический прогон по каталогу с тестовыми файлами, запускает кодирование и декодирование для каждого файла и формирует итоговую таблицу статистики.

ВЫВОД

В ходе лабораторной работы был реализован алгоритм № 4 – нумерационное кодирование на основе арифметического кодирования. Были написаны отдельные программы кодирования и декодирования, обеспечивающие полное восстановление исходных данных без потерь.

В базовой версии алгоритм использует композицию всего сообщения и арифметическое кодирование последовательности при известной композиции. Такой подход позволяет строить корректный архиватор без потерь и получать устойчивое сжатие на всех файлах тестового корпуса.

Далее были реализованы два улучшения. Первое улучшение позволило более компактно кодировать заголовок архива. Второе улучшение заключалось в переходе к блочному нумерационному кодированию, которое лучше учитывает локальные изменения статистики по файлу.

Результаты показали, что оба улучшения оказываются полезными, однако наибольший эффект даёт именно блочная версия. Для неё был получен минимальный суммарный размер сжатых файлов — 1_772_956 байт по всему Calgary.